Intune Win32 Launcher 2.1

The Intune Win32 Launcher is developed for system engineers responsible for deploying their Win32 applications using Microsoft Intune or other deployment tool. Microsoft Intune is a cloud-based endpoint management solution. It manages user access to organizational resources and simplifies app and device management across your many devices, including mobile devices, desktop computers, and virtual endpoints..

Some of the key benefits are:

- Intune simplifies app management with a built-in app experience, including app deployment, updates, and removal. You can connect to and distribute apps from your private app stores, enable Microsoft 365 apps, deploy Win32 apps, create app protection policies, and manage access to apps & their data.
- Intune automates policy deployment for apps, security, device configuration, compliance, conditional access, and more. When the policies are ready, you can deploy these policies to your user groups and device groups. To receive these policies, the devices only need internet access.
- Intune integrates with mobile threat defense services, including Microsoft Defender for Endpoint and third party partner services. With these services, the focus is on endpoint security and you can create policies that respond to threats, do real-time risk analysis, and automate remediation.
- You use a web-based admin center that focuses on endpoint management, including data-driven reporting. Admins can sign into the Intune admin center from any device that has internet access. This admin center uses Microsoft Graph REST APIs to programmatically access the Intune service. Every action in the admin center is a Microsoft Graph call. If you're not familiar with Graph, and want to learn more, go to Graph integrates with Microsoft Intune.
- The Microsoft Intune Suite offers advanced endpoint management and security. The suite has optional add-on features, including Remote Help, Endpoint Privilege Management, Microsoft Tunnel for MAM, and more.

Although Microsoft Intune has a lot to offer when it comes to Win32 apps deployment, user notification is not always possible or sufficient when a required application is installed or removed. To ensure an application is successfully installed or removed the first time right we have to make sure the following rules are in place:

- Establish if the end-user is logged on to the system.
- Determine if a pending reboot is active on the system.
- Notify the end-user if an installation or removal of an application will be performed.
- The end-user will have time to save his or her work before an installation or removal will take place.
- The end-user will have the option to defer the installation or removal of an application.
- The end-user is notified to keep the system active during installing or removing an application.
- The end-user is notified to close the application that is being updated or removed.
- All user processes that affect the installation or removal of an application can be stopped.
- Notify the end-user when an installation or removal of an application is finished.
- Configure the end-user profile information if needed during installing or removing an application.

This tool can be used as a replacement for the PSAppDeployToolkit to solve some of the compatibility issues when deploying Win32 applications using Microsoft Intune.

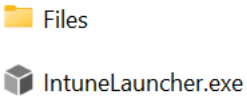
Provolve IT B.V. - Kalvermarkt 53 - 2511 CB - The Netherlands
+31 70 82 00 360 - info@provolve.nl - www.provolve.nl
VAT Number NL853876174B01 - Chambre of Commerce Number 60362421

Provolve IT

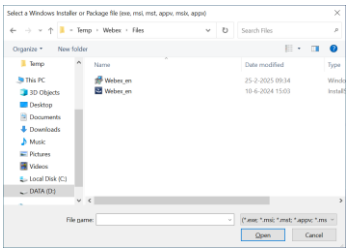
Contents

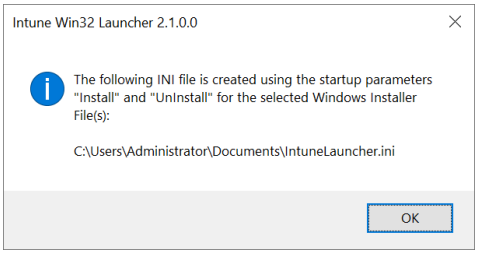
How to use

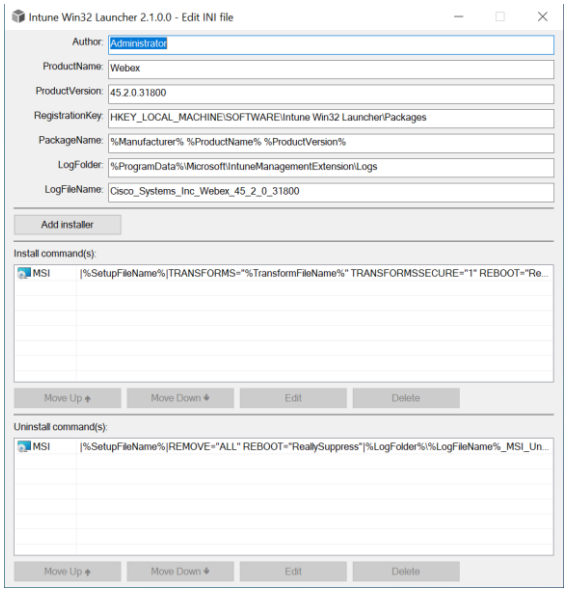
The Intune Win32 Launcher is a standalone executable available in a 64-bit release for running on a Microsoft Windows 10 or higher operating system. The executable is named *IntuneLauncher.exe* by default and will need its own corresponding INI file with the same name and must be in the same folder. The *Files* folder can be used for containing the supported Win32 setup installation or package files.



Use the ***IntuneLauncher.exe*** executable file together with the ***Files*** subfolder as shown above for each Win32 application package that needs to be deployed using Microsoft Intune or other deployment tool. When creating a Win32 application package, copy the setup (source) files in the ***Files*** subfolder and start the *IntuneLauncher.exe.* Select the main Windows Installer or Package file (*.exe, *.msi, *.mst, *.appv, *.appx or *.msix) that is present in the *Files* subfolder. The ***IntuneLauncher.ini*** configuration file will then be created automatically according to the chosen installer or package format. When adding an MSI-file with a transform (*.MST) file, select the MST-file first and select the corresponding MSI-File after.

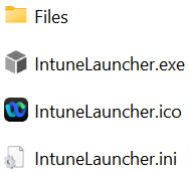




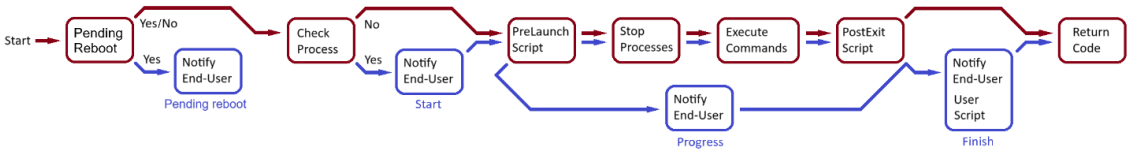


After the ***IntuneLauncher.ini*** configuration file is created you will be able to change some of the default variables and add or remove additional installer files. This will ensure that all required installer files are part of the Win32 application package and it will ensure the installer files are installed and removed in the right order. When you are done, close this Window and click yes for saving the ***IntuneLauncher.ini*** configuration file.

Optionally, it is possible to add the ***IntuneLauncher.ico*** or ***IntuneLauncher.png*** for changing the icon of the notification, progress and message windows for the end-user.

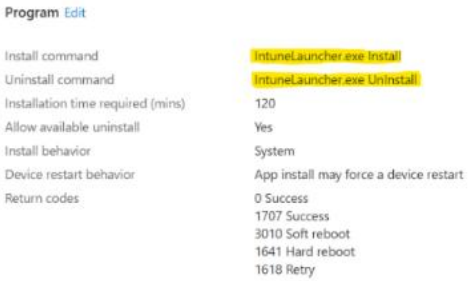


When performing an installation or removal the Intune Win32 Launcher must be started with elevated administrative privileges or the local system account. The Intune Win32 Launcher runtime will always use the following flow chart when performing an installation or removal of a Win32 application. The red line indicates the path flow when no notification is active or no end-user is logged on to the system and the blue line will indicate the path flow where the end-user is logged on and the notification is active.



Implementing the Intune Win32 Launcher

For implementing the Intune Win32 Launcher, edit the **IntuneLauncher.ini** file if needed and test the installation and removal before creating the *.intunewin file using the Microsoft Win32 Content Prep Tool. From the Intune deployment manager, the **IntuneLauncher.exe** can then be uploaded and started using the desired execution section as a startup parameter that needs to be available in the *IntuneLauncher.ini* file.



Optionally, the following second parameters can be added after the first *Install* or *UnInstall* parameter:

- **I** or **Interactive**
  Adding i or interactive after the first Install or UnInstall parameter will ensure that all the Win32 Application system processes started by the **IntuneLauncher.exe** will be visible for the logged in user during the installation or removal of the Win32 Application. Be aware that the user also is able to interact with the installation or removal. It is advised to use this option only during testing of the Win32 Application deployment or if user interaction is absolutely required to finish the installation or removal.

  Example: IntuneLauncher.exe Install i

INI File usage

As a best practice use the **Install** and **UnInstall** starting execution section names for performing both the installation or removal of the Win32 application package. The Intune Win32 Launcher executable can then use the Install or UnInstall execution sections as a startup parameter when performing the installation or removal using Microsoft Intune or other deployment tool. There are six section types available that can be used:

1- **Variables**
   The variables section is optional and the name is fixed with the name *Variables*. This section is used to set custom session environment variables that can be used for the key values or parts of these key values that are used in the sections described below.
2- **Execution**
   The execution sections will be used for starting the installation or removal of the Win32 application package. Make sure that the *SectionType* key name with the value *Execution* exists in this section.
3- **Notification**
   The notification sections can be called from the execution sections using the NotifyStart key name and will be used for notifying the end-user showing a graphical user interface before an installation or removal of the Win32 application will take place. Make sure that the *SectionType* key name with the value *Notification* exists in this section.
4- **Progress**
   The progress sections can be called from the execution sections using the NotifyProgress key name and will be used for notifying the end-user showing a graphical user interface during an installation or removal of the Win32 application. Make sure that the *SectionType* key name with the value *Progress* exists in this section.
5- **Message**
   The message sections can be called from the execution sections using the NotifyPendingReboot and NotifyFinish key name or the *Notify* script item when it is called from the script sections. This will be used for notifying the end-user showing a message box before, during or after an installation or removal of the Win32 application is finished. Make sure that the *SectionType* key name with the value *Message* exists in this section.
6- **Script**
   The script sections can be called from the execution sections using the PreLaunchScript and PostExitScript key names. A user script can also be executed using the UserScript key name from the Message notification sections. Do not use the SectionType key name in these sections. Script sections will use a different format for executing script items.

Variables section

The variable section is used when the Intune Win32 Launcher is started. This section is optional and the name is fixed. If the **Variables** section is not present, no custom session environment variables will be set. The Intune Win32 Launcher will read all variable names and values that will be set as a session environment variable during startup. These environment variables can then be used for the other section key values where the same value or part of that value is used more than once. You can define all your own variables in this section that will be started with the variable name, an equal sign (=) and the variable value.

Example:

```
[Variables]
Author=Administrator
Date=18-5-2026
Manufacturer=Cisco Systems, Inc
ProductName=Webex
ProductVersion=45.2.0.31800
ProductCode={72E15AE0-12FB-5462-A038-A19C5E4E523E}
PackageName=%Manufacturer% %ProductName% %ProductVersion%
RegistrationKey=HKEY_LOCAL_MACHINE\SOFTWARE\Intune Win32 Launcher\Packages
LogFolder=%ProgramData%\Microsoft\IntuneManagementExtension\Logs
LogFileName=Cisco_Systems_Inc_Webex_45_2_0_31800
SetupFileName=Webex_en.msi
TransformFileName=Webex_en.mst
```

For language support the default variable section **0409** can be used for all the default English text and link information that is shown during the notification for the end-user. The 0409 is the default English language code that will be used if the system default language code is 0409 or not found during runtime. You can add other language code sections with the appropriate translation values for supporting multiple languages. (See **Language Codes.pdf**)

Example:

```
[0409]
InformationTextNotifyPendingReboot=Pending reboot detected. Please restart your system first to continue this installation.
InstallInformationTextNotifyStart=is about to be installed on this system. Save your work and close this application when it is active.
UnInstallInformationTextNotifyStart=is about to be removed. Save your work and close this application when it is active.
InstallTimerTextNotifyStart=The installation will start in (mm:ss):
UnInstallTimerTextNotifyStart=The removal will start in (mm:ss):
InstallInformationTextNotifyProgress=is being installed. Do not turn off your computer during the installation. Please wait...
UnInstallInformationTextNotifyProgress=is being removed from this system. Do not turn off your computer during the removal. Please wait...
InstallInformationTextNotifyFinish=The installation of the software is successfully completed.
UnInstallInformationTextNotifyFinish=The removal of the software is successfully finished.
DeferLinkText=Defer
ContinueLinkText=Continue
CloseLinkText=Close
```

The following session environment variables will always be present during runtime, even if the Variables section is not used:

%WorkingDir%	The full path name value where the Intune Win32 Launcher is started from.
%ProgramsDir%	The full path and name of the Programs folder in the (all-users) Start Menu.
%StartMenuDir%	The full path and name of the (all-users) Start Menu folder.
%DesktopDir%	The full path and name of the folder containing the (all-users) desktop files.
%StartupDir%	The full path and name of the (all-users) Startup folder in the Start Menu folder.
%RebootRequired%	Will be set to 1 if a pending reboot is detected, otherwise it will be 0.
%FoundProcess%	The number of processes found running that are set in the CheckProcess key name.
%UserLogin%	Will be set to 1 if a user is logged on to the system, otherwise it will be 0.
%ReturnCode%	Will contain the return code value for the lastly executed command (Run).
%LanguageCode%	Will contain the system's default language code.

Execution sections

The execution section is used as a starting point for the Intune Win32 Launcher executable. Use the following key names with the desired values for both the Install and UnInstall **execution** sections.

SectionType
Use the value *Execution* to indicate this section is used as a startup execution section for the key names described below.
LogFile
Specify the full path with the log file name to use when this section is executed. If the LogFile property value is omitted, the default path will be %TEMP%\IntuneLaucher.log. If the log file path does not exist the full path will be created automatically if the provided drive exists on the system. See Troubleshooting for more information.
WorkingDir
This is the relative working directory for the Win32 application package files. If omitted, the path containing the *IntuneLauncher.exe* will be used. As a best practice use the value **Files** and make sure the Win32 application package files are stored in the subfolder Files.
SetSourceDir
Specify the full path for the source directory that will be used for copying the content from the folder specified in the *WorkingDir key name*. This path will then be used for installing (or removing) the Win32 application using the *Command* options. The SetSourceDir key name is optional. The folder specified in the *WorkingDir* key name will be used if the SetSourceDir key name is not specified. Keep in mind that the folder specified in the *WorkingDir* key name is always temporary and will be removed after the Intune Agent is done performing the installation or removal of the Win32 application.
DeleteSourceDir
If set to value 1, the source directory specified in the *SetSourceDir* key name will be removed after the installation or removal of the Win32 application is finished. For best practice, set this value to 1 only for removing (UnInstall) the Win32 application.
TaskKill
All executable process names separated by a comma (,) that need to be stopped before executing the command lines. The processes will only be stopped if they are running. If multiple processes are running with the same name, all processes using this name will be stopped. The TaskKill key name is optional. No processes will be stopped if it is omitted.
Command1
The first command line to be executed that will be used together with the created *WorkingDir* value path. Make sure the command line contains the target executable name together with the unattended command line switches to ensure that the target is executed unattended (silent). When using an MSI (with MST) file use |<MSIfile>|TRANSFORMS=<TransformFile>|<LogFile> for directly installing or removing a Windows Installer file. You can use *Powershell* commands if you start the command line with "**>**". Use " **\n** " for each Powershell command line or refer to a *Powershell* script (*.ps1).
Command2
The second command line to be executed. You can continue adding more *Command* options like Command3, Command4, etc. The exit code for the last command line will be used as the return code for Intune or other deployment tool. The variable %ReturnCode% will be set containing the last return code value.

CheckProcess
The names of the user processes separated by a comma (,) to be checked if it is running when the Win32 application installation or removal takes place. If omitted, or the processes are not found running, no user notification will take place before, during and after the installation or removal of the Win32 application. User notification will only be available if one or more specified process names are activated by the end-user and the key name values *NotifyPendingReboot*, *NotifyStart*, *NotifyProgress* and *NotifyFinish* are used. Specify *explorer.exe* if you always want to notify the end-user if he or she is logged on to the system.

NotifyPendingReboot
The name of the notification section that will be used for creating the notification window if a pending reboot is detected when the Intune Win32 Launcher is started and one or more process names specified in the key name value CheckProcess are active. Make sure the notification section exists and has the *Message* value for the *SectionType* key name. After displaying the NotifyPendingReboot message the Intune Win32 Launcher will stop with the exit code 350 canceling the installation or removal of the Win32 application. The NotifyPendingReboot key name is optional. If omitted, no pending reboot check and notification will take place. See Notification section for more information.

NotifyStart
The name of the notification section that will be used for creating the notification window before the installation or removal of the Win32 application takes place and one or more process names specified in the key name value *CheckProcess* are active. Make sure the notification section exists and has the *Notification* value for the *SectionType* key name. The NotifyStart key name is optional. If omitted, no user notification will take place. See Notification section for more information.

NotifyProgress
The name of the progress section that will be used for creating the progress window during the installation or removal of the Win32 application. Make sure the progress section exists and has the *Progress* value for the *SectionType* key name. The NotifyProgress key name is optional. If omitted, no user notification will take place. See Progress section for more information.

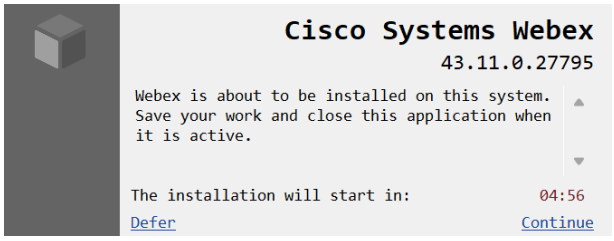
NotifyFinish
The name of the message section that will be used for creating the message window after the installation or removal of the Win32 application is finished. Make sure the message section exists and has the *Message* value for the *SectionType* key name. The NotifyFinish key name is optional. If omitted, no user notification will take place. See Message section for more information.

PreLaunchScript
The name of the *PreLaunch* script section that will be executed before the taskkill options and executing the command lines. The PreLaunchScript key name is optional. No pre-launch script will be performed if this is omitted. See PreLaunch, PostExit and User Script Sections for more information.

PostExitScript
The name of the *PostExit* script section that will be executed after executing the command lines. The PostExitScript key name is optional. No Post-Exit script will be performed if this is omitted. See PreLaunch, PostExit and User Script Sections for more information.

Notification sections

All notification windows will be displayed in the lower right corner of the primary monitor for the end user that is logged on to the system. The notification section is used to create a notification window for the end-user that will be logged on to the system before the installation or removal of the Win32 application will take place and one or more process names specified in the key name value *CheckProcess* are active. The information shown by the notification window can be adjusted by changing the key name values in the notification section.



Use the following key names with the desired values for the **notification** sections:

SectionType

Use the value *Notification* to indicate this section is used as a notification section for the key names described below.

ProductNameText

The product name shown in the notification window. As a best practice use the name of the Win32 application package that will be installed or removed. For example: *Cisco Systems Webex*

ProductVersionText

The product version shown in the notification window. As a best practice use the version of the Win32 application package that will be installed or removed. For example: *43.11.0.27795*

InformationText

The information text to be shown in the notification window. This text can contain all the information about the Win32 application package that will be installed or removed. For example: *Cisco Systems Webex is about to be installed on this system. Save your work and close this application when it is active.*

TimerText

The line of text shown before the timer. For example: *The installation will start in:*

DeferLinkText

The link name for deferring the installation or removal of the Win32 application package when the end-user will click on it. For example: *Defer*

ContinueLinkText

The link name for continuing the installation or removal of the Win32 application package when the end-user will click on it before the timer ends. For example: *Continue*

DeferTimes

The number of times the defer link will be available. The end-user will be able to defer the installation or removal of the Win32 application until this number is reached. The defer link will then not be available anymore. If omitted or 0, the defer link will not be available.

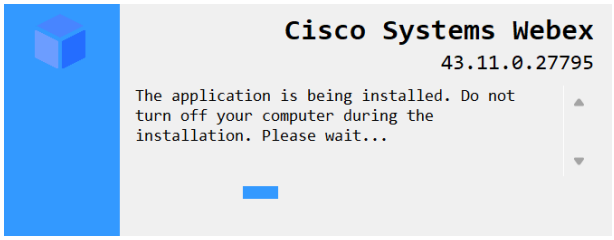
Wait

The number of seconds the notification windows will be active. If omitted, it will default to 60 seconds or 1 minute. The number of seconds will count down and will be shown in the time format MM:SS.

When the notification window is shown the end-user can reject the installation or removal of the Win32 application by clicking the Defer link if this is made available. This will temporarily cancel the installation or removal with the return code: 1602 (User cancelled installation.)

Progress sections

The progress section is used to create a progress window for the end-user that will be logged on to the system where the installation or removal of the Win32 application is taking place. The appearance of the progress window can be adjusted by changing the key name values in the progress section.



Use the following key names with the desired values for the **progress** sections:

SectionType

Use the value *Progress* to indicate this section is used as a progress section for the key names described below.

ProductNameText

The product name shown in the notification window. As a best practice use the name of the Win32 application package that will be installed or removed. For example: *Cisco Systems Webex*

ProductVersionText

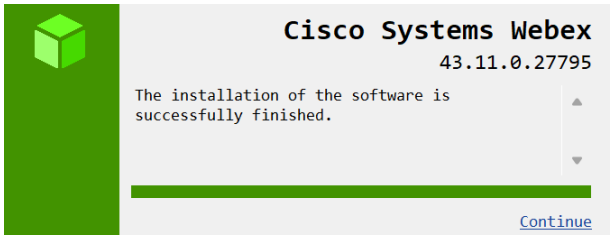
The product version shown in the notification window. As a best practice use the version of the Win32 application package that will be installed or removed. For example: *43.11.0.27795*

InformationText

The information text to be shown in the notification window. This text can contain all the information about the Win32 application package that will be installed or removed. For example: *The application is being installed. Do not turn off your computer during the installation. Please wait...*

Message sections

The message section is used to create a message window for the end-user that will be logged on to the system where the installation or removal of the Win32 application is finished or when the *Notify* script item is used. The appearance of the message window can be adjusted by changing the key name values in the message section.



Use the following key names and values for the **message** section:

SectionType

Use the value *Message* to indicate this section is used as a message section for the key names described below.

ReturnCodes

All the return codes separated by a comma (,) that will be used to show the message for the *InformationText* key name value. If the lastly executed command has a return code that is not present in the *ReturnCodes* key name value a yellow warning message windows will be shown instead, showing the message that is assigned to that return code. If omitted, the value *0,1707* will be used.

ProductNameText

The product name shown in the notification window. As a best practice use the name of the Win32 application package that will be installed or removed. For example: *Cisco Systems Webex*

ProductVersionText

The product version shown in the notification window. As a best practice use the version of the Win32 application package that will be installed or removed. For example: *43.11.0.27795*

InformationText

The information text to be shown in the notification window if the lastly executed command has a return code that is present in the *ReturnCodes* key name value. This text can contain all the information about the Win32 application package that is successfully installed or removed. For example: *The installation of the software is successfully finished.*

Status

If omitted, the window will display the green notification window indicating a successful message when the return code match the return codes present in the *ReturnCodes* key value. Otherwise, it will display a yellow notification window indicating a warning message. Use “Warning” to always display a yellow notification windows message or “Error” to display a red notification windows message indicating something went wrong.

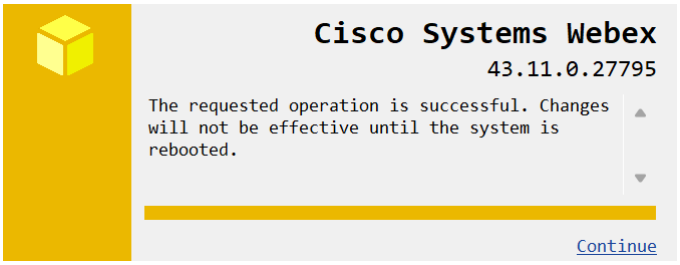
UserScript

The name of the *User* script section that will be executed when the message notification window is active and the lastly executed command has a return code that matches one of the return codes that is available in the *ReturnCodes* key name value. All available script items will be started under the user context of the user that is logged on to the system. Keep in mind that some script items will not be available and permissions are limited

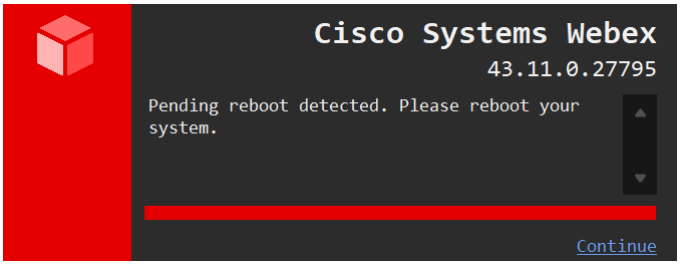
when executing script items. The UserScript key name is optional. No User script will be performed if this is omitted. See PreLaunch, PostExit and User Script Sections for more information.

Wait
The number of seconds the message windows will be active (after the UserScript is executed). If omitted, it will default to 30 seconds (after the UserScript is finished).

When the return code of the lastly executed command does not match the return codes available in the *ReturnCodes* key name, a yellow notification window will be displayed with the translated message for the received return code. The following example will display the return code that matches 3010.



When “Dark Mode” is enabled for the end-user that is logged on to the system, all the notification windows will be adjusted to the dark mode theme. The following example will display a notify pending reboot message where the dark mode theme is activated for the end-user.



Example:

```
──────────────────────────────┤ Intune Win32 Launcher 2.1.0.0 Configuration file ├──────────────────────────────

[Variables]
Author=Administrator
Date=18-5-2026
Manufacturer=Cisco Systems, Inc
ProductName=Webex
ProductVersion=45.2.0.31800
ProductCode={72E15AE0-12FB-5462-A038-A19C5E4E523E}
PackageName=%Manufacturer% %ProductName% %ProductVersion%
RegistrationKey=HKEY_LOCAL_MACHINE\SOFTWARE\Intune Win32 Launcher\Packages
LogFolder=%ProgramData%\Microsoft\IntuneManagementExtension\Logs
LogFileName=Cisco_Systems_Inc_Webex_45_2_0_31800
SetupFileName=Webex_en.msi
TransformFileName=Webex_en.mst

[0409]
InformationTextNotifyPendingReboot=Pending reboot detected. Please restart your system first to continue this installation.
InstallInformationTextNotifyStart=is about to be installed on this system. Save your work and close this application when it is active.
UnInstallInformationTextNotifyStart=is about to be removed. Save your work and close this application when it is active.
InstallTimerTextNotifyStart=The installation will start in (mm:ss):
UnInstallTimerTextNotifyStart=The removal will start in (mm:ss):
InstallInformationTextNotifyProgress=is being installed. Do not turn off your computer during the installation. Please wait...
UnInstallInformationTextNotifyProgress=is being removed from this system. Do not turn off your computer during the removal. Please wait...
InstallInformationTextNotifyFinish=The installation of the software is successfully completed.
UnInstallInformationTextNotifyFinish=The removal of the software is successfully finished.
DeferLinkText=Defer
ContinueLinkText=Continue
CloseLinkText=Close

[0413]
InformationTextNotifyPendingReboot=In afwachting van opnieuw opstarten. Graag zelf uw computer opnieuw opstarten zodat deze installatie doorgang krijgt.
InstallInformationTextNotifyStart=wordt zo meteen geïnstalleerd op dit systeem. Sluit de betreffende applicatie af indien deze actief is.
UnInstallInformationTextNotifyStart=wordt zo meteen verwijderd van dit systeem. Sluit de betreffende applicatie af indien deze actief is.
InstallTimerTextNotifyStart=De installatie begint na (mm:ss):
UnInstallTimerTextNotifyStart=Het verwijderen begint na (mm:ss):
InstallInformationTextNotifyProgress=wordt geïnstalleerd. Zorg ervoor dat dit systeem actief blijft gedurende de installatie. Even geduld a.u.b...
UnInstallInformationTextNotifyProgress=wordt verwijderd. Zorg ervoor dat dit systeem actief blijft gedurende het verwijderen. Even geduld a.u.b...
InstallInformationTextNotifyFinish=De installatie is succesvol afgerond.
UnInstallInformationTextNotifyFinish=Het verwijderen is succesvol afgerond.
DeferLinkText=Uitstellen
ContinueLinkText=Doorgaan
CloseLinkText=Sluiten

──────────────────────────────┤ Installation by starting: IntuneLauncher.exe Install ├──────────────────────────────
[Install]
SectionType=Execution
LogFile=%LogFolder%\%LogFileName%_Install.log
WorkingDir=Files
SetSourceDir=%ProgramData%\Package Cache\%ProductCode%
DeleteSourceDir=0
TaskKill=
Command1=|%SetupFileName%|TRANSFORMS="%TransformFileName%" TRANSFORMSSECURE="1"
REBOOT="ReallySuppress"|%LogFolder%\%LogFileName%_MSI_Install.log
CheckProcess=explorer.exe
NotifyPendingReboot=NotifyPendingReboot
```

```
NotifyStart=NotifyStartInstall
NotifyProgress=NotifyProgressInstall
NotifyFinish=NotifyFinishInstall
PreLaunchScript=
PostExitScript=Check_Install

[Check_Install]
ifEqual, ReturnCode, 0 OR 1707 OR 1641 OR 3010, Register_Install

[Register_Install]
RegWrite, REG_DWORD, %RegistrationKey%, %PackageName%, 1

[NotifyPendingReboot]
SectionType=Message
ProductNameText=%ProductName%
ProductVersionText=%ProductVersion% | %Manufacturer%
InformationText=%InformationTextNotifyPendingReboot%
ContinueLinkText=%CloseLinkText%
Status=Error
UserScript=
Wait=300

[NotifyStartInstall]
SectionType=Notification
ProductNameText=%ProductName%
ProductVersionText=%ProductVersion% | %Manufacturer%
InformationText=%ProductName% %InstallInformationTextNotifyStart%
TimerText=%InstallTimerTextNotifyStart%
DeferLinkText=%DeferLinkText%
ContinueLinkText=%ContinueLinkText%
DeferTimes=3
Wait=300

[NotifyProgressInstall]
SectionType=Progress
ProductNameText=%ProductName%
ProductVersionText=%ProductVersion% | %Manufacturer%
InformationText=%ProductName% %InstallInformationTextNotifyProgress%

[NotifyFinishInstall]
SectionType=Message
ReturnCodes=0,1707
ProductNameText=%ProductName%
ProductVersionText=%ProductVersion% | %Manufacturer%
InformationText=%InstallInformationTextNotifyFinish%
ContinueLinkText=%CloseLinkText%
Status=
UserScript=
Wait=60

──────────────────────────────┤ Uninstallation by starting: IntuneLauncher.exe UnInstall ├──────────────────────────────
[UnInstall]
SectionType=Execution
LogFile=%LogFolder%\%LogFileName%_UnInstall.log
WorkingDir=Files
SetSourceDir=%ProgramData%\Package Cache\%ProductCode%
DeleteSourceDir=1
TaskKill=
Command1=|%SetupFileName%|REMOVE="ALL" REBOOT="ReallySuppress"|%LogFolder%\%LogFileName%_MSI_UnInstall.log
CheckProcess=explorer.exe
NotifyPendingReboot=
NotifyStart=NotifyStartUnInstall
NotifyProgress=NotifyProgressUnInstall
NotifyFinish=NotifyFinishUnInstall
PreLaunchScript=
PostExitScript=Check_UnInstall
```

```
[Check_UnInstall]
ifEqual, ReturnCode, 0 OR 1707 OR 1641 OR 3010, Register_UnInstall

[Register_UnInstall]
RegWrite, REG_DWORD, %RegistrationKey%, %PackageName%, 0

[NotifyStartUnInstall]
SectionType=Notification
ProductNameText=%ProductName%
ProductVersionText=%ProductVersion% | %Manufacturer%
InformationText=%ProductName% %UnInstallInformationTextNotifyStart%
TimerText=%UnInstallTimerTextNotifyStart%
DeferLinkText=%DeferLinkText%
ContinueLinkText=%ContinueLinkText%
DeferTimes=3
Wait=300

[NotifyProgressUnInstall]
SectionType=Progress
ProductNameText=%ProductName%
ProductVersionText=%ProductVersion% | %Manufacturer%
InformationText=%ProductName% %UnInstallInformationTextNotifyProgress%

[NotifyFinishUnInstall]
SectionType=Message
ReturnCodes=0,1707
ProductNameText=%ProductName%
ProductVersionText=%ProductVersion% | %Manufacturer%
InformationText=%UnInstallInformationTextNotifyFinish%
ContinueLinkText=%CloseLinkText%
Status=
UserScript=
Wait=60
```

PreLaunch, PostExit and User Script Sections

Before and after launching the taskkill options and command lines for installing or removing the Win32 application it is possible to execute the following script items:

FileCopy, FileMove, FolderCopy, FolderMove, FolderCreate, FileDelete, FolderDelete, Set, EnvSet, RegImport, RegRead, RegWrite, RegDelete, FileVersion, IniRead, IniWrite, IniDelete, FileCreateShortcut, Run, MSIRemove, StringReplace, FileAppend, Download, Powershell, ServiceState, ServiceStart, ServiceStop, ServiceCreate, ServiceDelete, CreateTask, DeleteTask, Notify, ExitApp, Reboot and FileAccess.

All script items that are defined under the script sections will be executed starting at the top until the end of the script section is reached. Start each line under the *PreLaunchScript, PostExitScript* and/or *UserScript* script section name with the script item name and a comma (,) followed by a number of item values also separated by a comma. The number of item values and what item values that are used are depending on the script item used as described below.

Script Item	Item Values									Show Description
FileCopy	Source (wildcard) Pattern	Destination (wildcard) Pattern	1 *(overwrites)* 0 *(do not overwrite)*							UserScript description
FileMove	Source (wildcard) Pattern	Destination (wildcard) Pattern	1 *(overwrites)* 0 *(do not overwrite or rename)*							UserScript description
FolderCopy	Source Folder	Destination path	1 *(overwrites)* 0 *(do not overwrite)*							UserScript description
FolderMove	Source Folder	Destination path	R *(rename)* 2 *(overwrite with no limitation)* 1 *(overwrites)* 0 *(do not overwrite)*							UserScript description
FolderCreate	Folder name path									UserScript description
FileDelete	File(s) (wildcard) Pattern									UserScript description
FolderDelete	Folder name path (Deletes all files and subfolder)									UserScript description
Set	Name of the session environment variable	Value for the session environment variable. If omitted, the environment variable is deleted.								UserScript description
EnvSet	Name of the system or user environment variable	Value for the system or user environment variable. If omitted, the environment variable is deleted.								UserScript description
RegImport	Register filename path (*.reg)	0 or omitted *(Do not translate the environment variables inside the registry file to their actual values)* 1 *(Translate all environment variables inside the registry file to their actual values)*								UserScript description
RegRead	HKEY_LOCAL_MACHINE HKEY_CURRENT_USER	Value name *(Value name and value will be set as a session Environment Variable)*	Default Value							UserScript description
RegWrite	REG_SZ, REG_EXPAND_SZ, REG_MULTI_SZ, REG_DWORD, REG_BINARY, REG_QWORD	HKEY_LOCAL_MACHINE HKEY_CURRENT_USER	Value name	Value *(If the value doesn't exist the default value will be created)*						UserScript description
RegDelete	HKEY_LOCAL_MACHINE HKEY_CURRENT_USER	Value name (optional)								UserScript description
FileVersion	Executable filename path *(FileVersion name and value will be set as a session Environment Variable)*									UserScript description
IniRead	INI Filename	Section name	Key name *(Key name and value will be set a session Environment Variable)*	Default Value						UserScript description
IniWrite	Value	INI Filename	Section name	Key name						UserScript description
IniDelete	INI Filename	Section name	Key name (optional)							UserScript description
FileCreateShortcut	Target	Link File Name.lnk	WorkingDir	Args	Description	IconFile	ShortcutKey	Icon Number	RunState 1 Normal 3 Maximized 7 Minimized	UserScript description
Run	Target *(On exit the return code value will be set as the same value for the session environment variable name RetunCode)*	WorkingDir	Min, Max or Hide *(If omitted, the target executable launches normally)*	0 (wait for target command to end) 1 (the target command will be started without waiting for it to end)						UserScript description
MSIRemove	ProductCode (Windows installer unique MSI identifier)	LogFile (Log file name path for verbose logging)								UserScript description

StringReplace	Source Filename *(If Source and Destination filename are equal, multiple String Replacements can be done for the same file and Overwrite will be ignored)*	Name of the text string to find	Name of the text or variable value to replace with *(All occurrences of the search text will be replaced)*	Destination filename *(If the file doesn't exist the file will be created first)*	1 *(overwrites)* 0 *(Skip StringReplace when destination file already exists)*	UserScript description
FileAppend	Text write to the end of a file	Full file name path *(If the file doesn't exist the file will be created first)*	UTF-8, UTF-8-RAW, UTF-16 or UTF-16-RAW *(If omitted the default encoding will be ANSI)*			UserScript description
Download	URL (https/ftp)	Full file name path *(Any existing file will be overwritten)*				UserScript description
Powershell	Powershell command or script (*.ps1). Use " \n " (*newline*) between each separate Powershell command line.	WorkingDir				UserScript description
ServiceState	ServiceName *(Service name and status as the value Stopped, Start Pending, Stop Pending, Running, Continue Pending, Pause Pending, Paused or Unknown will be set as a session Environment Variable)*					UserScript description
ServiceStart	ServiceName (This script item will not be available for UserScript)					
ServiceStop	ServiceName (This script item will not be available for UserScript)					
ServiceCreate	ServiceName (This script item will not be available for UserScript)	BinaryPath (Path with Arguments)	StartType "Automatic" or "OnDemand" ("Disabled" when empty)	DisplayName (optional)	StartName (optional) DomainName\UserName Or the local system account will be used when empty.	Password (optional)
ServiceDelete	ServiceName (This script item will not be available for UserScript)					

CreateTask	TaskName (This script item will not be available for UserScript)	Task description	Path (executable)	Arguments	WorkingDir	Limited execution runtime in hours (default is 12)	TriggerType: "Logon" or "Boot"	Battery (When empty the task will only start when the system is connected to a power supply)
DeleteTask	TaskName (This script item will not be available for UserScript)							
Notify	The name of the notification section that needs to be executed *(This script item will not be available for UserScript)*							
ExitApp	The exit- or return code that will be given back to the (Intune) deployment manager. (If omitted this will be 0) *(This script item will not be available for UserScript)*							
Reboot	The exit- or return code that will be given back to the (Intune) deployment manager. (If omitted this will be 350) *(This script item will not be available for UserScript)*							
FileAccess	FileName path for setting read and execute permissions for everyone. *(This script item will not be available for UserScript)*							

*Existing (Session) Environment Variables (%) can be used instead of fixed texts for file (paths) and registry values. The **UserScript description** will show the entered text inside the message notification windows when performing the user script item.*

Use the following If (/ Else) statement items to execute a separate script section that can be started on a specific value of an existing (session) environment variable or whether or not a specified file, folder (path), registry key exist or a specified service or process is running or not. You can use the following statement items:

IfEqual, IfNotEqual, IfGreater, IfGreaterOrEqual, IfNotGreater, IfNotGreaterOrEqual, IfSmaller, IfSmallerOrEqual, IfNotSmaller, IfNotSmallerOrEqual, IfExist, IfNotExist, IfProcessExist, IfNotProcessExist, IfTaskExist and IfNotTaskExist.

Start a line under the *PreLaunchScript, PostExitScript* and/or *UserScript* script section name with the statement item name and a comma (,) followed by a number of item values also separated by a comma. The number of item values and what item values that are used are depending on the statement item used as described below.

Script Item	Item Values				Show Description
IfEqual	The name of the (session) environment variable value	The check value for the environment variable value *(Use **OR** between multiple values)*	The name of the script section that needs to be executed if one or more value(s) of the environment variable name is **equal** to the check value	The name of the script section that needs to be executed if one or more value(s) of the environment variable name is **not equal** to the check value *(This item is optional)*	UserScript description
IfNotEqual	The name of the (session) environment variable value	The check value for the environment variable value *(Use **OR** between multiple values)*	The name of the script section that needs to be executed if one or more value(s) of the environment variable name is **not equal** to the check value	The name of the script section that needs to be executed if one or more value(s) of the environment variable name is **equal** to the check value *(This item is optional)*	UserScript description
IfGreater	The name of the (session) environment variable value that contains a (version) number value	The check value for the environment variable value	The name of the script section that needs to be executed when the value of the environment variable name is **greater** than the check value	The name of the script section that needs to be executed when the value of the environment variable name is **smaller** than, or **equal** to the check value *(This item is optional)*	UserScript description
IfNotGreater	The name of the (session) environment variable value that contains a (version) number value	The check value for the environment variable value	The name of the script section that needs to be executed when the value of the environment variable name is **smaller** than, or **equal** to the check value	The name of the script section that needs to be executed when the value of the environment variable name is **greater** than the check value *(This item is optional)*	UserScript description

IfGreaterOrEqual	The name of the (session) environment variable value that contains a (version) number value	The check value for the environment variable value	The name of the script section that needs to be executed when the value of the environment variable name is **greater** than, or **equal** to the check value	The name of the script section that needs to be executed when the value of the environment variable name is **smaller** than the check value *(This item is optional)*	UserScript description
IfNotGreaterOrEqual	The name of the (session) environment variable value that contains a (version) number value	The check value for the environment variable value	The name of the script section that needs to be executed when the value of the environment variable name is **smaller** than the check value	The name of the script section that needs to be executed when the value of the environment variable name is **greater** than, or **equal** to the check value *(This item is optional)*	UserScript description
IfSmaller	The name of the (session) environment variable value that contains a (version) number value	The check value for the environment variable value	The name of the script section that needs to be executed when the value of the environment variable name is **smaller** than the check value	The name of the script section that needs to be executed when the value of the environment variable name is **greater** than, or **equal** to the check value *(This item is optional)*	UserScript description
IfNotSmaller	The name of the (session) environment variable value that contains a (version) number value	The check value for the environment variable value	The name of the script section that needs to be executed when the value of the environment variable name is **greater** than, or **equal** to the check value	The name of the script section that needs to be executed when the value of the environment variable name is **smaller** than the check value *(This item is optional)*	UserScript description
IfSmallerOrEqual	The name of the (session) environment variable value that contains a (version) number value	The check value for the environment variable value	The name of the script section that needs to be executed when the value of the environment variable name is **smaller** than, or **equal** to the check value	The name of the script section that needs to be executed when the value of the environment variable name is **greater** than the check value *(This item is optional)*	UserScript description
IfNotSmallerOrEqual	The name of the (session) environment variable value that contains a (version) number value	The check value for the environment variable value	The name of the script section that needs to be executed when the value of the environment variable name is **greater** than the check value	The name of the script section that needs to be executed when the value of the environment variable name is **smaller** than, or **equal** to the check value *(This item is optional)*	UserScript description
IfExist	File, folder (path) or registry key name *(Use **AND/OR** between multiple values)*	The name of the script section that needs to be executed when the file, folder (path) or registry key name statement **exists**	The name of the script section that needs to be executed when the file, folder (path) or registry name statement do **not exist** *(This item is optional)*		UserScript description
IfNotExist	File, folder (path) or registry key name *(Use **AND/OR** between multiple values)*	The name of the script section that needs to be executed when the file, folder (path) or registry name statement do **not exists**	The name of the script section that needs to be executed when the file, folder (path) or registry name statement **exist** *(This item is optional)*		UserScript description
IfProcessExist	The Process (ID) name	The name of the script section that needs to be executed when the process name or process ID is running	The name of the script section that needs to be executed when the process name or process ID is **not running** *(This item is optional)*		UserScript description
IfNotProcessExist	The Process (ID) name	The name of the script section that needs to be executed when the process name or process ID is **not** running	The name of the script section that needs to be executed when the process name or process ID is **running** *(This item is optional)*		UserScript description
IfTaskExist	The task name	The name of the script section that needs to be executed when the task name exist	The name of the script section that needs to be executed when the task name do **not exist** *(This item is optional)*		UserScript description
IfNotTaskExist	The task name	The name of the script section that needs to be executed when the task name does **not** exist	The name of the script section that needs to be executed when the task name **exist** *(This item is optional)*		UserScript description

*The **UserScript description** will show the entered text inside the message notification windows when performing the statement item for the user script.*

Use the script items **RegRead**, **IniRead, FileVersion** or **ServiceState** to get a specified registry value, key value from an .ini file, file version or the status of a Windows Service that will be set as a session environment variable for the If- and IfNot- statement items to execute a separate script section depending on the registry value, INI file key item value or file version. It is also possible to use the existing session environment variables **RebootRequired**, **UserLogin, FoundProcess** or **ReturnCode** to execute a separate script section depending on the value.

Script sections started from the “**Execution**” SectionType under the “PreLaunchScript” and “PostExitScript” properties will always be started with elevated administrative privileges or the local system account. Script sections started from the “**Message**” SectionType under the “UserScript” property will always be started under the user that is logged on to the system. If no user is logged on, the UserScript sections will not be executed. You can check if a user is logged on to the system if the %UserLogin% session environment variable is set to 1. The following script example will exit the installation or removal before starting using the return code 1317 (The specified account does not exist) if no user is logged on to the system:

```
PreLaunchScript=CheckUserLogin
PostExitScript=

[CheckUserLogin]
IfEqual, UserLogin, 0, Abort

[Abort]
ExitApp, 1317
```

Script Items

We will explain each script item in detail on how to use them.

FileCopy

Copies one or more files.

- Source pattern

  The name of a single file or folder, or a wildcard pattern such as C:\Temp*.tmp. SourcePattern is assumed to be in the working directory if an absolute path isn't specified. You can also use the %WorkingDir% variable that will be the starting path where the Intune Launcher is started from.

- Destination pattern

  The name or pattern of the destination, which is assumed to be in the working directory if an absolute path isn't specified. The destination directory must already exist. Environment variables can be used like %ALLUSERSPROFILE%\MyDir.

- Overwrite

  This parameter determines whether to overwrite files if they already exist. If this parameter is **1** (true), the command overwrites existing files. If omitted or **0** (false), the command does not overwrite existing files.

FileMove

Moves or renames one or more files.

- Source pattern

  The name of a single file or folder, or a wildcard pattern such as C:\Temp*.tmp. SourcePattern is assumed to be in the working directory if an absolute path isn't specified. You can also use the %WorkingDir% variable that will be the starting path where the Intune Launcher is started from.

- Destination pattern

  The name or pattern of the destination, which is assumed to be in the working directory if an absolute path isn't specified. The destination directory must already exist. Environment variables can be used like %ALLUSERSPROFILE%\MyDir.

- Overwrite

  This parameter determines whether to overwrite files if they already exist. If this parameter is **1** (true), the command overwrites existing files. If omitted or **0** (false), the command does not overwrite existing files.

The following example will rename a single file:
FileMove, C:\File Before.txt, C:\File After.txt

FolderCopy

Copies a folder along with all its sub-folders and files (similar to xcopy) or the entire contents of an archive file such as ZIP.

- Source

  Name of the source directory (with no trailing backslash), which is assumed to be in the Intune Launcher working directory if an absolute path isn't specified. You can also use the %WorkingDir% variable that will be the starting path where the Intune Launcher is started from. For example: %WorkingDir%:\Files\Folder1

- Destination

  Name of the destination directory (with no trailing backslash), which is assumed to be in the Intune Launcher working directory if an absolute path isn't specified. Environment variables can be used.

- Overwrite

  This parameter determines whether to overwrite files if they already exist. Specify one of the following values:

  •**0** = (false) Do not overwrite existing files. The operation will fail and have no effect if Dest already exists as a file or directory.

  •**1** = (true) Overwrite existing files. However, any files or subfolders inside Dest that do not have a counterpart in Source will not be deleted.

FolderMove

Moves a folder along with all its sub-folders and files. It can also rename a folder.

- Source

  Name of the source directory (with no trailing backslash), which is assumed to be in the Intune Launcher working directory if an absolute path isn't specified. You can also use the %WorkingDir% variable that will be the starting path where the Intune Launcher is started from. For example: %WorkingDir%:\Files\Folder1

- Destination

  Name of the destination directory (with no trailing baskslash), which is assumed to be in the Intune Launcher working directory if an absolute path isn't specified. Environment variables can be used.

- Overwrite

  This parameter determines whether to overwrite files if they already exist. If omitted, the default value will be “R” for renaming the folder. Specify one of the following values:

  •**0** = (false) Do not overwrite existing files. The operation will fail if Dest already exists as a file or directory.

  •**1** = (true) Overwrite existing files. However, any files or subfolders inside Dest that do not have a counterpart in Source will not be deleted. Known limitation: If Dest already exists as a folder and it is on the same volume as Source, Source will be moved into it rather than overwriting it. To avoid this, see the next option.

  •**2** = The same as mode 1 above except that the limitation is absent.

  •**R** = Rename the directory rather than moving it. Although renaming normally has the same effect as moving, it is helpful in cases where you want "all or none" behavior; that is, when you don't want the operation to be only partially successful when Source or one of its files is locked (in use). Although this method cannot move Source onto a different volume, it can move it to any other directory on its own volume. The operation will fail if Dest already exists as a file or directory.

The following example will rename a single folder:
FolderMove, C:\My Folder, C:\My Folder (renamed), R

FolderCreate

Creates a folder.

- DirName

  Name of the directory to create, which is assumed to be in the Intune Win32 Launcher working directory if an absolute path isn't specified. Environment variables can be used like %ALLUSERSPROFILE%\DirName

FileDelete

Deletes one or more files.

- FilePattern

  The name of a single file or a wildcard pattern such as C:\Temp*.tmp. FilePattern is assumed to be in the Intune Launcher working directory if an absolute path isn't specified.

FolderDelete

Deletes a folder.

- DirName

  Name of the directory to delete, which is assumed to be in the Intune Win32 Launcher working directory if an absolute path isn't specified. This will remove all files and subdirectories (like "rmdir /S").

Set

Writes a value to a variable contained in the running session for the Intune Win32 Launcher process or deletes the session variable. If this script item is started under the system account (default) the variable will be set for the Intune Win32 Launcher process running under the system. If this script item is started under the end-user using the *UserScript* option the variable will be set for the Intune Win32 Launcher process running under the current user.

- Var

  Name of the session environment variable to use, e.g. "ReturnCode".

- Value

  Value to set the session variable to. If omitted, the session environment variable is deleted.

EnvSet

Writes a value to a variable contained in the system or user environment or deletes the system or user environment variable. If this script item is started under the system account (default) the variable will be set for the system. If this script item is started under the end-user using the *UserScript* option the variable will be set as a user variable for the current user.

- EnvVar

  Name of the system or user environment variable to use, e.g. "COMSPEC" or "PATH".

- Value

  Value to set the system or user environment variable to. If omitted, the system or user environment variable is deleted.

System account: The variable will be set as a system environment variable located in:
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\Session Manager\Environment
User account: The variable will be set as a user environment variable located in:
HKEY_CURRENT_USER\Environment

RegImport

Imports a registry file for adding and/or removing registry keys and values from a register file (*.reg).

- FileName

  Specify the registry filename (*.reg) which is assumed to be in the Intune Win32 Launcher working directory if an absolute path isn't specified.

- Transform

  This parameter determines if environment variables inside de register file needs to be translated.
  •0 = (false) Do not translate environment variables to their actual values. (default when empty)
  •1 = (true) Translate environment variables to their actual values.

When importing a registry file in a User Script, first make sure the registry file can be accessed by everyone using FileAccess first.

RegRead

Reads a value from the registry and set the value name and value as a session environment variable. You can use the If- and IfNot- statement items to execute a separate script section depending on the registry value.

- KeyName

  The full name of the registry key. This must start with HKEY_LOCAL_MACHINE, HKEY_USERS, HKEY_CURRENT_USER, HKEY_CLASSES_ROOT, or HKEY_CURRENT_CONFIG (or the abbreviations for each of these, such as HKLM).

- ValueName

  The name of the value to retrieve. Together with the value itself this will be set as a session environment variable that can be used for the PreLaunchScript, PostExitScript or UserScript script actions or launching the Run or command line executables.

- DefaultValue

  The default value to return if the specified key or value does not exist.

RegWrite

Writes a value to the registry.

- ValueType

  Must be either REG_SZ, REG_EXPAND_SZ, REG_MULTI_SZ, REG_DWORD, REG_BINARY or REG_QWORD.

- KeyName

  The full name of the registry key. This must start with HKEY_LOCAL_MACHINE, HKEY_USERS, HKEY_CURRENT_USER, HKEY_CLASSES_ROOT, or HKEY_CURRENT_CONFIG (or the abbreviations for each of these, such as HKCU).

- ValueName

  The name of the value that will be written to. If blank or omitted, the key's default value will be used. The default value is displayed as "(Default)" by RegEdit.

- Value

  The value to be written. This can also be an environment variable like %TEMP% where the actual value will be used.

RegDelete

Deletes a value from the registry.

- KeyName

  The full name of the registry key. This must start with HKEY_LOCAL_MACHINE, HKEY_USERS, HKEY_CURRENT_USER, HKEY_CLASSES_ROOT, or HKEY_CURRENT_CONFIG (or the abbreviations for each of these, such as HKLM).

- ValueName

  The name of the value to delete. If blank or omitted, the registry subkey will be deleted.

FileVersion

Retrieves the version of a file and set the FileVersion name and value as a session environment variable. Use the If- and IfNot- statement items to execute a separate script section.

- FileName

  Specify the name (path) of the target (executable) file.

Use the following session environment variable to get the last retrieved file version:

%FileVersion%	Will contain the last retrieved file version.

IniRead

Reads a value from a standard format .ini file and set the key name and value as a session environment variable. Use the If- and IfNot- statement items to execute a separate script section.

- Filename

  The name of the .ini file, which is assumed to be in the Intune Win32 Launcher working directory if an absolute path isn't specified.

- Section

  The section name in the .ini file, which is the heading phrase that appears in square brackets (do not include the brackets in this parameter).

- Key

  The key name in the .ini file.

- Default

  If omitted, an Error is thrown on failure. Otherwise, specify the value to return on failure.

IniWrite

Writes a value or section to a standard format .ini file.

- Value

  The string or number that will be written to the right of Key's equal sign (=).

- Filename

  The name of the .ini file, which is assumed to be in the Intune Win32 Launcher working directory if an absolute path isn't specified.

- Section

  The section name in the .ini file, which is the heading phrase that appears in square brackets (do not include the brackets in this parameter).

- Key

  The key name in the .ini file.

IniDelete

Deletes a value from a standard format .ini file.

- Filename

  The name of the .ini file, which is assumed to be in the Intune Win32 Launcher working directory if an absolute path isn't specified.

- Section

  The section name in the .ini file, which is the heading phrase that appears in square brackets (do not include the brackets in this parameter).

- Key

  If omitted, the entire section will be deleted. Otherwise, specify the key name in the .ini file.

FileCreateShortcut

Creates a shortcut (.lnk) file.

- Target

  Name of the file that the shortcut refers to, which should include an absolute path unless the file is integrated with the system (e.g. Notepad.exe). The file does not have to exist at the time the shortcut is created; however, if it does not, some systems might alter the path in unexpected ways.

- LinkFile

  The location and name of the shortcut file to be created, which is assumed to be in %WorkingDir% if an absolute path isn't specified. Be sure to include the .lnk extension. The destination directory must already exist. If the file already exists, it will be overwritten. You can use the following starting location variables for creating the shortcut file.

%ProgramsDir%	The full path and name of the Programs folder in the (all-users) Start Menu.
%StartMenuDir%	The full path and name of the (all-users) Start Menu folder.
%DesktopDir%	The full path and name of the folder containing the (all-users) desktop files.
%StartupDir%	The full path and name of the (all-users) Startup folder in the Start Menu folder.

  If this script item is started under the system account (default) the “all-users” path location is used for the variables shown above. If this script item is started under the end-user using the *UserScript* option, the user path location is used for the variables shown above.

- WorkingDir

  If blank or omitted, LinkFile will have a blank "Start in" field and the system will provide a default working directory when the shortcut is launched. Otherwise, specify the directory that will become Target's current working directory when the shortcut is launched.

- Args

  If blank or omitted, Target will be launched without parameters. Otherwise, specify the parameters that will be passed to Target when it is launched. Separate parameters with spaces. If a parameter contains spaces, enclose it in double quotes.

- Description

  If blank or omitted, LinkFile will have no description. Otherwise, specify comments that describe the shortcut (used by the OS to display a tooltip, etc.)

- IconFile

  If blank or omitted, LinkFile will have Target's icon. Otherwise, specify the full path and name of the icon to be displayed for LinkFile. It must either be an .ICO file or the very first icon of an EXE or DLL file.

- ShortcutKey

If blank or omitted, LinkFile will have no shortcut key. Otherwise, specify a single letter or number. All shortcut keys are created as Ctrl+Alt shortcuts. For example, if the letter B is specified for this parameter, the shortcut key will be Ctrl+Alt+B.

- IconNumber

If omitted, it defaults to 1. Otherwise, specify the number of the icon to be used in IconFile. For example, 2 is the second icon.

- RunState

If omitted, it defaults to 1. Otherwise, specify one of the following digits to launch Target minimized or maximized:

•1 = Normal •3 = Maximized •7 = Minimized

An alternative way to create a shortcut to a URL is the following IniWrite example, which creates a special URL shortcut. Change the first two parameters to suit your preferences:

```
IniWrite, https://www.google.com, %ProgramsDir%\My Shortcut.url, InternetShortcut, URL
IniWrite, C:\Icons.dll, %ProgramsDir%\My Shortcut.url, InternetShortcut, IconFile
IniWrite, 0, %ProgramsDir%\My Shortcut.url, InternetShortcut, IconIndex
```

Run

Runs an external program and set the return code value to the session environment name RetunCode.

- Target

An executable file (.exe, .com, .bat, etc.), shortcut (.lnk), or system verb to launch. If Target is a local file and no path was specified with it, the working directory will be searched first. If no matching file is found there, the system will search for and launch the file if it is integrated ("known"), e.g. by being contained in one of the PATH folders. To pass parameters, add them immediately after the program or document name.

- WorkingDir

The working directory for the launched item. Do not enclose the name in double quotes even if it contains spaces. If omitted, the Intune Win32 Launcher working directory will be used. You can also use the %WorkingDir% variable that will be the starting path where the Intune Win32 Launcher is started from.

- Options

If omitted, the function launches Target normally. To change this behavior, specify one or more of the following words:
Max: launch maximized
Min: launch minimized
Hide: launch hidden (cannot be used in combination with either of the above).

- NoWait

This parameter determines whether to wait for the external program to end or immediately continue after the external program is started. Specify one of the following values:

•**0** = (false) Wait for the external program to end before continuing.
•**1** = (true) Immediately continue after the external program is started.

Use the following session environment variable to retrieve the exit- or return code of the lastly executed “Run” script item:

%ReturnCode% Will contain the return code value for the script item “Run”.

MSIRemove

Removes (Uninstall) a Windows Installer (MSI) product that is currently installed on the computer and set the return code value to the session environment name RetunCode.

- ProductCode

  The unique identifier for the particular Windows Installer product release, represented as a string GUID. For example: {12345678-1234-1234-1234-123456789012}

- LOG Filename

  The file name (including the full path) that will contain all the verbose information when the Windows Installer product is removed. Make sure the path location for the log file already exist. (optional)
  For example: %LogFolder%\MyProduct_MSI_UnInstall.log

StringReplace

Replaces the specified substring with a new string in a file.

- Source filename

  The full path to the file name that contains the file content with the specified substring that needs to be replaced. For example: %ProgramFiles%\ApplicationName\Config.cfg

- Search text

  The substring value to search for that needs to be replaced. The file content can contain one or multiple occurrences that needs to be replaced. For example: <ComSpec>

- Replace value

  The new value text to replace the search text with. All occurrences of the search text will be replaced. Environment variables can be used and will be replaced with the actual value. For example: %ComSpec%

- Destination filename

  The full path to the (non-existing) file path that needs to contain the new replaced file content. If the destination file doesn't exist the file will be created first. If the source and destination file names are equal, multiple string replacements with different search texts and replacement values can be performed for the same file and the overwrite option will be ignored. For example:
  %ProgramData%\ApplicationName\Config.cfg

- Overwrite

  This parameter determines whether to overwrite the destination file if it already exist. Specify one of the following values:
  •**0** (false): Do not overwrite the existing destination file. The operation will be ignored when the destination file already exists.
  •**1** (true): Overwrite the existing destination file. Even if the file already exists, the operation will be performed.

FileAppend

Writes text or binary data to the end of a file (first creating the file, if necessary).

- Text

  The text or raw binary data to append to the file. A linefeed characters (`n) will be added after each text or binary data added. Make sure a linefeed exists after the existing text or binary content to make sure the new text or binary data is added as a new line.

- Filename

  The name of the file to be appended, which is assumed to be in the Intune Win32 Launcher working directory if an absolute path isn't specified. The destination directory must already exist.

- Options

  Encoding: Specify any of the encoding names UTF-8, UTF-8-RAW, UTF-16 or UTF-16-RAW. If no encoding name is specified the default existing coding will be used.

Download

Downloads a file from the Internet.

- URL

  URL of the file to download. For example, http://someorg.org might retrieve the welcome page for that organization.

- Filename

  Specify the name of the file to be created locally, which is assumed to be in the Intune Win32 Launcher working directory if an absolute path isn't specified. Any existing file will be overwritten by the new file.

Powershell

Executes one or more Powershell commands or a Powershell script (*.ps1). If the command or script is returning information, this can be found in the log file. See Troubleshooting for more information.

- Command

  Enter the Powershell script or command line to be executed. If multiple command lines are being used make sure that each command line is separated by a space **\n** and another space.

  For example:
  Powershell, Add-AppVClientPackage -path '%SetupFileName%' \n Publish-AppVClientPackage -Global -PackageId %PackageId% -VersionId %VersionId%

- WorkingDir

  The current directory path Location for the PowerShell commands. Do not enclose the name in double quotes even if it contains spaces. If omitted, the Intune Win32 Launcher working directory will be used.

ServiceState

Retrieves the status from a Windows service that exists on the system and set the service name and status as a value that will be set as a session environment variable. Use the If- and IfNot- statement items to execute a separate script section.

- ServiceName

  Specify the name of the Windows service (do not use the display name).

The service status can be one of the following values:
Stopped, Start Pending, Stop Pending, Running, Continue Pending, Pause Pending, Paused or Unknown

The following example will get the status for the “Print Spooler” service:
ServiceState, Spooler

%Spooler%	Will contain the last retrieved status. (for example: Running)

ServiceStart

Start a Windows service that exists on the system.

- ServiceName

  Specify the name of the Windows service (do not use the display name).

The following example will start the “Print Spooler” service:
ServiceStart, Spooler

ServiceStop

Stop a Windows service that exists on the system.

- ServiceName

  Specify the name of the Windows service (do not use the display name).

The following example will stop the “Print Spooler” service:
ServiceStop, Spooler

ServiceCreate

Create a Windows service that does not exist on the system. (Not available for UserScript)

- ServiceName

  The name of the Windows service that will be created under HKLM\SYSTEM\CurrentControlSet\Services.
- BinaryPath

  The fully qualified path to the service binary file. The path can also include arguments for an auto-start service. For example, "d:\myshare\myservice.exe arg1 arg2"
- StartType

  Use "Automatic" (Auto) or "OnDemand" (Demand). If omitted the service will be created with the start type "Disabled".
- DisplayName

  The display name to be used by the user interface programs to identify the service. If omitted the serviceName will be used. (optional)
- StartName

  The name of the account under which the service should run. Use an account name in the form DomainName\UserName. The service process will be logged on as this user. If the account belongs to the built-in domain, you can specify .\UserName. If omitted, the Local System account will be used. (optional)
- Password

  The password to the account name specified by the *StartName* parameter. Specify an empty string if the account has no password. (optional)

For example: *ServiceCreate, Calc, C:\Windows\System32\calc.exe, Demand, My Test Service*

ServiceDelete

Delete a Windows Service that exists on the system. (Not available for UserScript)

- ServiceName

  Specify the name of the Windows Service (do not use the display name).

For example: *ServiceDelete, Calc*

CreateTask

Creates a scheduled task running under the system account that will be set for execution when someone is logging on to the system (next logon) or when the system is booted (next boot). (Not available for UserScript)

- TaskName

  The name of the scheduled task.
- Description

  The description of the scheduled task.
- Path

  The path to the executable on the system that will be executed.
- Argument

  The arguments for starting the executable. (optional)
- WorkingDir

  The working directory for the executable. (optional)
- ExecutionTimeLimit

  The number of hours the executable is allowed to run. If omitted, this will be set to 12 hours.
- TriggerType

  Use **Logon** (default when omitted) or **Boot** for the task to start at the next system boot or next logon.
- Battery

  When empty, the task will only start when a power supply is connected. Use any value (like 1 or on) to also start the task when the system is running on battery power.

Use the **ifTaskExist** or **ifNotTaskExist** statement items to check whether the task name already exists or not.

DeleteTask

Deletes a scheduled task that was created using the CreateTask script item. (Not available for UserScript)

- TaskName

  The name of the scheduled task.

Use the **ifTaskExist** or **ifNotTaskExist** statement items to check whether the task name already exists or not.

Notify

Will show a message notification window when a user is logged on to the system. (Not available for UserScript)

- Message section

  The message section is used to create a message window for the end-user that will be logged on to the system. See Message sections for more information.

ExitApp

Terminates the Intune Win32 Launcher with a specified exit code. (Not available for UserScript)

- ExitCode

  Specify the return- or exitcode that will be send back to the (Intune) deployment manager. If omitted, the return- or exit code will be 0. (The operation completed successfully.)

Reboot

Terminates the Intune Win32 Launcher with a specified exit code and then reboots the system. (Not available for UserScript)

- ExitCode

  Specify the return- or exitcode that will be send back to the (Intune) deployment manager. If omitted, the return- or exit code will be 1641. (The requested operation completed successfully. The system will be restarted so the changes can take effect.)

Use the session environment variable “RebootRequired” to determine if a pending reboot is currently active and use the session environment variable “UserLogin” to determine if a user is currently logged on to the system.

%RebootRequired%	Will be set to 1 if a pending reboot is detected, otherwise it will be 0.
%UserLogin%	Will be set to 1 if a user is logged on to the system, otherwise it will be 0.

FileAccess

Will set read and execute permissions for everyone on the specified file. (Not available for UserScript)

- File Name

  The filename (including the path) for setting read and execute permissions for everyone. This will make the file accessible for copying or executing the file for the end-user when performing the UserScript.

All script items that are defined under the script section will be set starting at the top until the end of the script section is reached. Script items set under the *PreLaunchScript* section name will be executed for the system before the Win32 application is installed or removed and script items set under the *PostExitScript* section name will be executed for the system after the Win32 application is installed or removed. Script items set under the *UserScript* section will be executed for the end-user where the notification message window is shown and the lastly executed command line matches one of the return codes in the *RetunCodes* key name value of the message notification section.

Troubleshooting

When the Intune Win32 Launcher executable is started a log file will be created. The log file location can be set using the **LogFile** key property in the execution section in the *IntuneLauncher.ini* file. The log file will provide all the information during runtime. The logfile contains a line buildup starting at the top with a timestamp followed by an *Info*, *Warning* or *Error* level statement and details on the performed action that involves executing the script items, stopping processes and starting the command lines.

Example logfile:

Timestamp	Level	Details
20241217110551	[INFO]	Intune Win32 Launcher 1.9.0.0 started with Process Id 836 for section: Install
20241217110551	[INFO]	Computer name: WIN-11E-NL-X64
20241217110551	[INFO]	Operating System version: 10.0.22631 (64-bit=1)
20241217110551	[INFO]	Operating System language code: 0413
20241217110551	[INFO]	User name: SYSTEM
20241217110551	[INFO]	RebootRequired: 0
20241217110551	[INFO]	UserLogin: 0
20241217110551	[INFO]	Working directory: C:\Windows\IMECache\095b1b5a-d0bb-4688-95d3-26dd06079196_9
20241217110551	[INFO]	INI file: C:\Windows\IMECache\095b1b5a-d0bb-4688-95d3-26dd06079196_9\IntuneLauncher.ini
20241217110551	[INFO]	[Performing Executing section: Install]
20241217110551	[INFO]	CheckProcess: explorer.exe
20241217110551	[INFO]	explorer.exe exists with Process Id: 6068
20241217110551	[INFO]	FoundProcess: 1
20241217110551	[INFO]	[Activating Notification Process]
20241217110555	[INFO]	CreateProcessAsUser function is started for User01 with Process Id: 6520
20241217110555	[INFO]	[Performing Notification section: NotifyStartInstall]
20241217111056	[INFO]	Notification process returned: 0
20241217111056	[INFO]	[Performing Notification section: NotifyProgressInstall]
20241217111057	[INFO]	Notification process returned: 0
20241217111057	[INFO]	Using MSI PackagePath: C:\Windows\IMECache\095b1b5a-d0bb-4688-95d3-26dd06079196_9\Package Files\Webex_en.msi
20241217111057	[INFO]	Using MSI CommandLine: TRANSFORMS=Webex_en.mst TRANSFORMSSECURE=1
20241217111057	[INFO]	Using MSI LogFile: C:\ProgramData\Microsoft\IntuneManagementExtension\Logs\Cisco_Systems_Inc_Webex_44_9_0_30650_MSI_Install.log
20241217111057	[INFO]	Starting MsiInstallProduct...
20241217111130	[INFO]	MsiInstallProduct successfully stopped with return code: 0
20241217111130	[INFO]	[Performing PostExit script section: Check_Install]
20241217111130	[INFO]	Using IfEqual statement for variable name 'ReturnCode' with value '0 OR 1641 OR 3010'. When true, goto script section 'Register_Install'. Else, continue...
20241217111130	[INFO]	[Performing script section: Register_Install]
20241217111130	[INFO]	RegWrite create 'REG_DWORD' registry item 'Cisco Systems, Inc Webex 44.9.0.30650' in 'HKEY_LOCAL_MACHINE\SOFTWARE\Intune Win32 Launcher' with value '1'
20241217111130	[INFO]	RegWrite successfully executed.
20241217111130	[INFO]	[Performing Notification section: NotifyFinishInstall]
20241217111231	[INFO]	Notification process returned: 0
20241217111231	[INFO]	Intune Win32 Launcher stopped successfully. (Return Code = 0)
----------	---------	----------

Powershell commands that return message information can also be found but will be logged without the timestamp or level information. For example:

```
20250806142306          [ INFO ]    [ Performing script section: EnablePackageScripts ]
20250806142306          [ INFO ]    Execute Powershell command: Set-AppvClientConfiguration -EnablePackageScripts 1
20250806142306          [ INFO ]    Working directory:
20250806142307          [ INFO ]    Powershell command successfully stopped (return code: 0) with return message:

Name                Value SetByGroupPolicy
----                ----- ----------------
EnablePackageScripts 1       False


20250806142307          [ INFO ]    Continuing without TaskKill...
```

If interaction is needed when the Intune Win32 Launcher is started from session 0 (default when using the Intune Win32 system deployment) add “I” or “Interactive” as an extra parameter after the execution section parameter. This option will launch a second (SYSTEM) Intune Win32 Launcher process in the logged on user session showing all graphical user interfaces (GUI’s) started under the SYSTEM account.

Example: IntuneLauncher.exe Install i

Microsoft Defender Exploit Guard

Because the Intune Win32 Launcher is a third party tool that is developed in Autohotkey (v2) and compiled as an executable the Microsoft Defender Exploid Guard can block the execution for a number of reasons. One reason is the possibility that the executable may not be signed by default. Signing the executable can be done using the signtool.exe and a Personal Information Exchange (. pfx) file. It is also possible to compile the Intune Win32 Launcher tool yourself using the source code and the Ahk2Exe.exe tool that will be present on your system when installing Autohotkey (v2). (www.autohotkey.com).

The Microsoft Defender Exploid Guard can also block the Intune Win32 Launcher and this has to do with the fact that the notification windows are started from the initial Intune Win32 Launcher process running under the local system account that will use the **CreateProcessAsUser** function. This action can be seen as a malicious action because it can potentially run code for the end-user that is logged on.

```
20240119180620  [ INFO  ]       Intune Win32 Launcher 1.2.0.0 started with Process Id 7304 for section: Install
20240119180620  [ INFO  ]       Computer name: XXX-XXXXXXXXX
20240119180620  [ INFO  ]       Operating System version: 10.0.19045 (64-bit=1)
20240119180620  [ INFO  ]       Operating System language code: 0413
20240119180620  [ INFO  ]       User name: SYSTEM
20240119180620  [ INFO  ]       Working directory: C:\Windows\IMECache\095b1b5a-d0bb-4688-95d3-26dd06079196_6
20240119180620  [ INFO  ]       INI file: C:\Windows\IMECache\095b1b5a-d0bb-4688-95d3-26dd06079196_6\IntuneLauncher.ini
20240119180620  [ INFO  ]       [ Performing Executing section: Install ]
20240119180620  [ INFO  ]       CheckProcess: explorer.exe
20240119180620  [ INFO  ]       explorer.exe exists with Process Id: 8128
20240119180620  [ INFO  ]       [ Performing Notification section: NotifyStartInstall ]
20240119180620  [ ERROR ]       CreateProcessAsUser function stopped with error: -2. Intune Win32 Launcher is stopped.
--------------  ---------       ------------------------------------------------------------------------------------
```

Although the Intune Win32 Launcher does not have malicious intensions, the ability to perform an installation or perform basic script actions that are defined in the *IntuneLauncher.ini* file can also be seen as malicious when it is executed on the system.

Consult with your security department for using the Intune Win32 Launcher tool. An exception for the IntuneLauncher.exe file may be necessary for running this tool.

Developer information

My name is Ferry van Gelderen and I am an IT Professional that has more than 25 years of experience in the Microsoft application packaging/virtualization and deployment field. In 2014, together with my companion Dereck Breuning, we started the company Provolve IT and we started with the release of the “Easy Software Deployment” application that I started to develop some years prior. Developing tools to make our live easier is always a fun thing to do especially when you know this will help others move forward in their IT quest.

This software is provided "as is". Use of the software is free and at your own risk.
This software is created in AutoHotKey v2.0 and the source code is available for review and/or adjustments.

Learn more about Intune
https://learn.microsoft.com/en-us/mem/intune/fundamentals/what-is-intune

Learn more about the Microsoft Win32 Content Prep Tool
https://learn.microsoft.com/en-us/mem/intune/apps/apps-win32-prepare

Learn more about AutoHotKey:
https://www.autohotkey.com

Contact information:
Name: Ferry van Gelderen
Email: Ferry@Provolve.nl
Website: https://provolve.nl/